

DELTA — Autonomous Financial Workflow & Agentic Commerce Engine on Arc

The Stablecoins Commerce Stack Challenge

Submitted for: Track 4 — Best Agentic Economy Experience on Arc *(Secondary alignment: Track 1 — Best Cross-Border Payments & Remittances)*

Host: Arc (Circle) & Ignyte

1. Executive Summary & Project Metadata

| Field | Details |

|---|---|

| **Project Title** | DELTA (Autonomous Financial Workflow Engine) |

| **Short Description** | An agentic financial orchestration platform built natively on Arc L1 and Circle's developer stack. Delta empowers AI agents and businesses to visually design, condition-gate, and execute automated multi-step financial workflows—ranging from cross-border remittances to automated SME payroll and CCTP bridging—with mathematical safety and real-time USDC settlement. |

| **Primary Track** | Track 4: Best Agentic Economy Experience on Arc |

| **Secondary Track** | Track 1: Best Cross-Border Payments & Remittances Experience (UAE → Global) |

| **Circle Account Email** | `keoyl52@gmail.com` |

| **GitHub Repository** | <https://github.com/keoyl52/delta> |

| **Video Demonstration** | <https://www.youtube.com/watch?v=p5U0WwiAHbA> |

| **Live Web App URL** | <https://delta-omega-black.vercel.app> |

| **Circle Products Used** | `USDC on Arc`, `Circle Developer-Controlled Wallets`, `CCTP / Bridge Kit`, `Circle EURC Token` |

2. Problem Statement & The Delta Solution

The Friction in Agentic & Cross-Border Commerce

Traditional financial workflows—such as vendor invoice release, cross-border contractor payouts, multi-currency treasury rebalancing, and agentic micro-purchases—are plagued by:

- 1. Manual Friction & Delay:** Human operational oversight required at every settlement boundary.
- 2. Single-Hop Fragility in AI Agents:** Existing AI agent frameworks execute single linear steps without multi-hop safety guards, creating catastrophic risks of double-spending or misrouting funds down un-evaluated paths.
- 3. High Settlement Overhead:** Legacy rails impose high fee unpredictability and slow settlement times.

The Delta Innovation

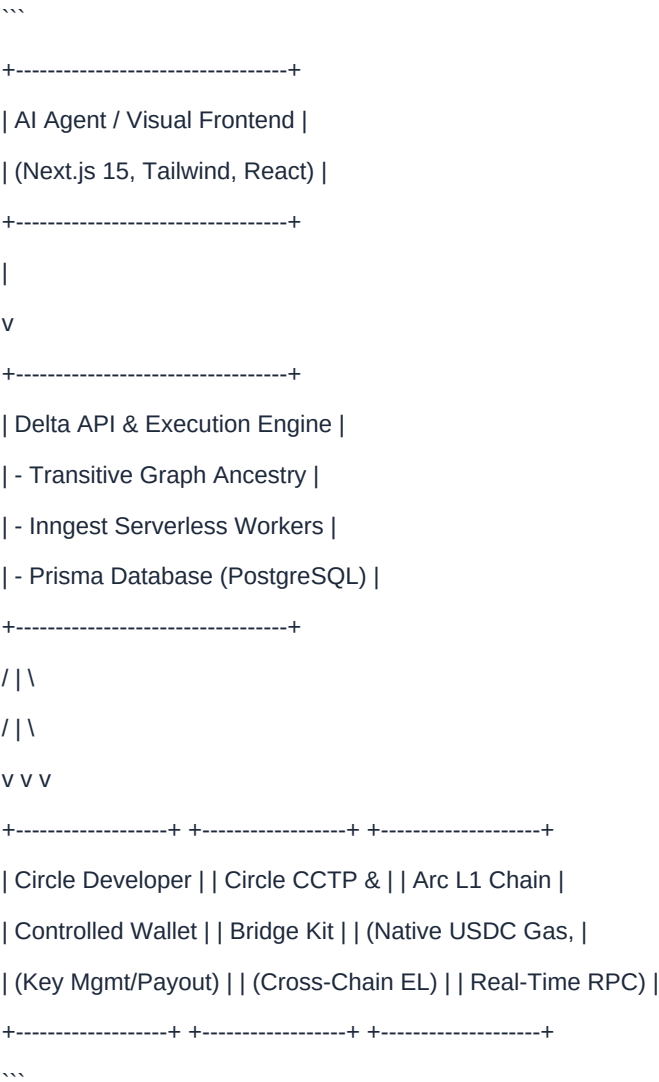
Delta turns complex financial commerce into **visually configurable, self-executing graph workflows** powered by Arc L1 and Circle's developer stack:

- **Visual Drag-and-Drop Workflow Canvas:** Users and AI agents construct graph pipelines with `Deposit`, `Schedule`, `Condition Gate`, `Swap`, `Bridge (CCTP)`, `Send`, `Hold`, and `Notify` nodes.

- **Transitive Path Gating Engine:** Delta introduces a transitive graph evaluation algorithm (`checkNodeAncestry`). Downstream execution nodes (`Send`, `Bridge`, `Swap`) strictly verify that all ancestor condition nodes evaluated to `TRUE` along their specific incoming execution path. If a condition evaluates to `FALSE`, the entire downstream sub-graph is safely `SKIPPED`, preventing unauthorized money movement.
- **Circle Native Stack Integration:** Powered by Circle Developer-Controlled Wallets for programmatic key management, CCTP Bridge Kit for zero-slippage cross-chain USDC movement, and Arc L1 for deterministic, dollar-denominated gas settlement.

3. System Architecture & Technical Deep-Dive

Architectural Layers



Component Breakdown

1. **Trigger Engine** (`Deposit`, `Schedule`, `Event`):
 - Webhook & event listeners monitor Circle Wallets and Arc L1 blocks for incoming USDC/EURC deposits or timed crons.
2. **Condition Gate Engine** (`Condition Gate`):
 - Dynamically evaluates expressions (e.g., `depositAmount >= 10 USDC`).
 - Generates deterministic execution handles (`TRUE` / `FALSE`).

3. Transitive Path Execution Safeguard (`checkNodeAncestry`):

- Resolves the complete ancestor graph for every target action node.
- Guarantees multi-hop safety: downstream nodes cannot be tricked by single-hop evaluation logic.

4. Settlement & Bridge Layer (`Send`, `Bridge`, `Swap`):

- Executes programmatic payouts using Circle Developer-Controlled Wallets.
- Triggers CCTP burn-and-mint via Circle Bridge Kit for cross-chain settlement (Arc ↔ Ethereum / Arbitrum / Avalanche / Solana).

4. Circle Products Integration Matrix

| Circle Product | Implementation Location in Delta | Role & Value Delivered |

|---|---|---|

| **USDC on Arc** | `src/lib/inngest/functions.ts` | Primary stablecoin settlement asset for all workflows, delivering deterministic dollar-denominated fees on Arc L1. |

| **Circle Developer-Controlled Wallets** | `src/app/api/wallet/` | Programmatic wallet generation, balance querying, and secure transaction signing without user private key exposure. |

| **CCTP & Bridge Kit** | `src/app/api/wallet/bridge/route.ts` | Zero-slippage cross-chain USDC transfer across Arc and foreign EVM/Solana networks. |

| **Circle EURC Token** | `src/app/api/wallet/withdraw/route.ts` | Multi-currency stablecoin support enabling EURC payouts and UAE/European remittance corridors. |

5. Circle Product Feedback *(Mandatory Section)*

Why We Chose Circle Products

Circle's developer infrastructure is the gold standard for programmable money. The combination of **Circle Developer-Controlled Wallets** for automated signing, **CCTP** for capital-efficient cross-chain movement, and **Arc L1** for dollar-denominated gas fees provided the exact foundation required for Delta's agentic workflow engine.

What Worked Exceptionally Well

1. **Developer-Controlled Wallets REST API:** Clean authentication flow and reliable transaction status polling. The SDK made it trivial to generate dedicated operational wallets for each workflow execution.
2. **Arc L1 Gas Predictability:** Eliminating native gas token volatility (paying gas natively in USDC) drastically simplified our workflow balance estimation and fee calculations.
3. **CCTP Bridge Kit Abstraction:** The Bridge Kit SDK abstracted away complex attestation polling and burn/mint event tracking into single clean function calls.

What Could Be Improved & Recommendations

1. **CCTP Testnet Attestation Latency:** During peak testnet activity, attestation signatures occasionally took 10–15 minutes. Exposing a WebSocket or Webhook subscription for attestation status (instead of HTTP polling) would improve real-time agentic UX.
2. **Graph Execution Helper SDKs:** Providing an official Circle TypeScript helper for tracking multi-hop transaction execution graphs (parent-child transaction chains) would benefit developers building complex agentic pipelines.
3. **EURC Faucet Availability on Arc Testnet:** Expanding automated EURC faucet availability on Arc Testnet would accelerate developer testing for multi-currency FX workflows.

6. Functional MVP & Real-World Use Cases

Demonstrated Use Cases in Delta

1. Agentic Micro-Purchases & Subscriptions (Track 4):

- AI agent receives a deposit on Arc -> checks budget condition -> executes real-time USDC payment via Circle Wallets -> notifies system via webhook.

2. Cross-Border Payroll & Contractor Remittance (Track 1):

- Business deposits USDC on Arc -> Condition Gate checks invoice verification -> CCTP bridges funds to contractor's preferred chain -> Settle in USDC/EURC.

3. Milestone-Based SME Escrow (Track 2):

- Import/export buyer deposits USDC -> Workflow holds funds -> Proof-of-delivery event triggers condition gate -> Automated release of funds to supplier.

7. How to Run Delta Locally

```
```bash
```

### 1. Clone repository

---

```
git clone https://github.com/keoyle52/delta.git
```

```
cd delta
```

### 2. Install dependencies

---

```
npm install
```

### 3. Environment setup

---

```
cp .env.example .env
```

**Set CIRCLE\_API\_KEY, CIRCLE\_ENTITY\_SECRET, and CIRCLE\_EURC\_TOKEN\_ID**

---

### 4. Run database migration & Prisma setup

---

```
npx prisma generate
```

```
npx prisma db push
```

### 5. Start development server & Inngest worker

---

```
npm run dev
```

**In a separate terminal: npx inngest-cli@latest dev**

---

...

---

\*Created with pride for The Stablecoins Commerce Stack Challenge on Arc L1.\*